Relatório 8: Conectividade Wi-Fi.
Internet das Coisas Integral
Prof. Gabriel de Freitas Fernandes
2026.2

EQUIPE:
Arthur Peixoto - 170.214
Gabriel Oliva Gurgel - 168.860
Guilherme Samuel dos Santos - 168.867

1 Introdução

Esta prática tem como objetivo implementar a conexão de um microcontrolador ESP32 à rede Wi-Fi para enviar dados de sensores e atuadores diretamente para uma planilha no Google Drive. No contexto de Internet das Coisas (IoT), essa integração é fundamental para monitorar informações de forma remota e automática.

Para atingir esse objetivo, utilizaram-se as bibliotecas de rede do ESP32 e uma ferramenta de integração com o Google Sheets. O sistema foi programado para escanear as redes Wi-Fi disponíveis, exibir a intensidade do sinal de cada uma, permitir a conexão interativa via terminal e registrar os dados coletados de forma periódica na planilha.

2 Prática

2.1 Montagem Experimental

O esquema eletrônico seguiu o mesmo que o estabelecido anteriormente com o módulo conversor analógico-digital externo **ADS1115**, um sensor LDR simples e um servomotor **MG996R**.

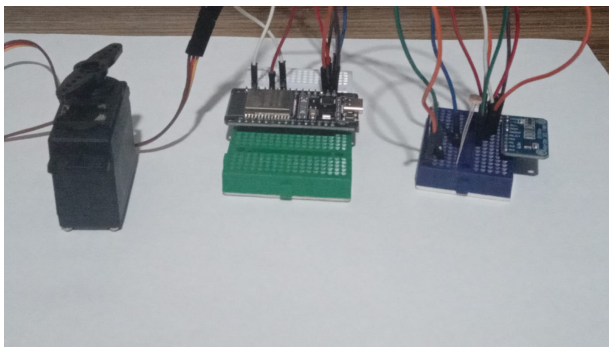


Figura 1: Versão feita de teste no pós-aula (sem capacitor)

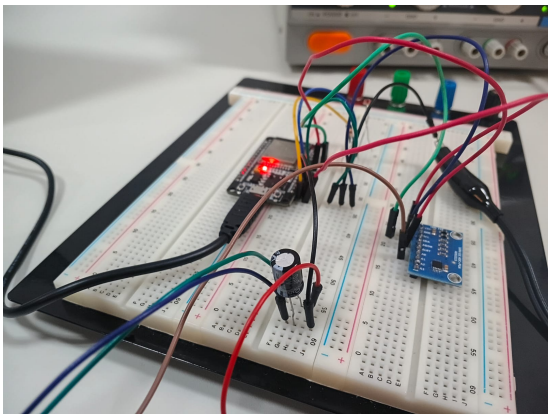


Figura 2: Versão feita de teste durante aula (com capacitor)

2.2 Código

O código desenvolvido na Arduino IDE para estabelecer a conectividade Wi-Fi e sincronizar os dados com a planilha eletrônica é apresentado na Listagem 1. As variáveis `dado1` e `dado2` representam os canais de dados utilizados para o envio.

Para questões de simplificação, a chave privada foi reduzida a uma *string* simples no exemplo a seguir. Ela está presente na versão de anexo.

```
#include <WiFi.h>
#include <ESP_Google_Sheet_Client.h>
#include <ESP32Servo.h>
#include <Adafruit_ADS1X15.h>
#include <Wire.h>

Adafruit_ADS1115 ads;
Servo servo;

#define MOTOR_PIN 18
#define PROJECT_ID "projeto-do-dragao-guerreiro"
#define CLIENT_EMAIL "esp32iot@projeto-do-dragao-guerreiro.iam.gserviceaccount.com"
#define SPREADSHEET_ID "1dnOn1AiQJhA3T02qwPbDCbBpHlYDftc1H0ZB63vDMWQ"
const char PRIVATE_KEY[] PROGMEM = "my-private-key";

bool leituraConcluida = false;
bool gsheetInitialized = false;

unsigned long ultimoEnvio = 0;
const unsigned long intervaloEnvio = 5000; // Intervalo de 5 segundos entre envios

// 3 variáveis com valores 1, 2 e 3
int dado1 = 1;
int dado2 = 2;
int dado3 = 3;

String ssid = "";
String password = "";

String readSerialLine() {
  while (Serial.available() == 0) {
    delay(50);
  }
  String input = Serial.readStringUntil('\n');
  input.trim();
  return input;
}

void conectarWifiPelaSerial() {
  while (WiFi.status() != WL_CONNECTED) {
    Serial.println("\n--- Internet Scan ---");
    int n = WiFi.scanNetworks();

    if (n == 0) {
      Serial.println("Nenhuma rede detectada.");
      delay(3000);
      continue;
    }

    Serial.print("Foram encontradas ");
    Serial.print(n);
    Serial.println(" redes:");
    for (int i = 0; i < n; ++i) {
      Serial.print(i + 1);
      Serial.print(": ");
      Serial.print(WiFi.SSID(i));
      Serial.print(" (");
      Serial.print(WiFi.RSSI(i));
      Serial.print(" dBm)");
      Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN) ? " [Open]" : " [Locked]"
    );
      delay(10);
    }

    // Limpa caracteres residuais no buffer da serial
    while (Serial.available() > 0) {
      Serial.read();
    }
```

```
68
69      Serial.println("\nQual o nome (SSID) da rede que pretende conectar?");
70      ssid = readSerialLine();
71      Serial.println("Rede selecionada: " + ssid);
72
73      Serial.println("Qual a senha dessa rede? Se for pública digite 0:");
74      password = readSerialLine();
75
76      Serial.print("A conectar a " + ssid);
77      if (password == "0" || password.length() == 0) {
78        WiFi.begin(ssid.c_str());
79      } else WiFi.begin(ssid.c_str(), password.c_str());
80
81      int t = 0;
82      while (WiFi.status() != WL_CONNECTED && t < 20) {
83        Serial.print(".");
84        delay(500);
85        t++;
86      }
87
88      if (WiFi.status() == WL_CONNECTED) {
89        Serial.println("\nConectado ao Wi-Fi com sucesso!");
90        Serial.print("IP ESP32: ");
91        Serial.println(WiFi.localIP());
92        Serial.print("RSSI: ");
93        Serial.print(WiFi.RSSI());
94        Serial.println(" dBm");
95      } else {
96        Serial.println("\nFalha ao conectar. Tente novamente.");
97        WiFi.disconnect();
98        delay(2000);
99      }
100   }
101 }
102
103 void adicionarLinha(int v1, int v2) {
104   FirebaseJson response;
105   FirebaseJson valueRange;
106
107   // 1. Cria a linha
108   FirebaseJsonArray row;
109   row.add(v1);
110   row.add(v2);
111
112   // 2. Cria a matriz de valores e adiciona a linha dentro dela
113   FirebaseJsonArray values;
114   values.add(row);
115
116   // 3. Atribui a matriz ao campo "values" do objeto principal
117   valueRange.set("values", values);
118
119   // Envia com append para a primeira aba
120   bool success = GSheet.values.append(&response, SPREADSHEET_ID, "Página1!A1", &
        valueRange);
121   /*if (success) {
122     Serial.println("Dados gravados com sucesso!");
123     response.toString(Serial, true);
124     Serial.println();
125     return;
126   } */
127
128   if(!success) {
129     Serial.print("Erro ao gravar: ");
130     Serial.println(GSheet.errorReason());
131   }
132 }
133
134 void setup() {
```

```
  Serial.begin(115200);
  delay(1000);
  WiFi.disconnect();
  WiFi.setAutoReconnect(true);
  // Executa o scan e a conexão interativa via Serial
  conectarWifiPelaSerial();
  // Inicia o processo de autenticação do Google Sheets
  Serial.println("\nA iniciar autenticação com o Google Sheets...");
  GSheet.begin(CLIENT_EMAIL, PROJECT_ID, PRIVATE_KEY);
  gsheetInitialized = true;
  servo.attach(MOTOR_PIN);
  Wire.begin();
  while(!ads.begin(0x48)) Serial.println("Failed to initialize.");
  Serial.println("Start ADS1115.");
}

void loop() {
  // Se perder conexão com a rede, reconecta
  if (WiFi.status() != WL_CONNECTED) {
    conectarWifiPelaSerial();
    return;
  }

  // Verifica continuamente a autenticação do token do Google
  bool ready = GSheet.ready();

  if (ready) {
    // 1. Lê os 3 primeiros dados da tabela
    if (!leituraConcluida) {
      FirebaseJson response;

      Serial.println("\nA obter valores da folha de cálculo...");
      Serial.println("---------------------------------------------------------------------");

      // Lê as células A1:A3 da aba Página1
      bool success = GSheet.values.get(&response, SPREADSHEET_ID, "Página1!A1:A3");

      if (success) {
        Serial.println("Dados obtidos com sucesso:");
        response.toString(Serial, true);
        Serial.println();
        leituraConcluida = true;
      } else {
        Serial.print("Erro ao ler planilha: ");
        Serial.println(GSheet.errorReason());
      }
    }
    // 2. Depois de ler, fica enviando continuamente a cada intervalo (5s)
    else {
      if (millis() - ultimoEnvio >= intervaloEnvio) {
        ultimoEnvio = millis();
        int16_t adc0;
        adc0 = ads.readADC_SingleEnded(0);
        Serial.print("ADC0 Raw: ");
        Serial.println(adc0);
        int angle = map(adc0, 13000, 17200, 0, 180);
        servo.write(angle);
        Serial.print("Servo Angle: ");
        Serial.println(angle);

        adicionarLinha((int)adc0, angle);
      }
    }
  }
}
```

Listing 1: Implementação da varredura Wi-Fi e envio de dados para o Google Sheets no ESP32.

2.2.1 Funcionamento da Conexão e Envio de Dados

O programa utiliza as bibliotecas de rede do ESP32 para gerenciar a conexão sem fio. Na inicialização, a função de varredura busca as redes Wi-Fi disponíveis, exibindo o nome de cada uma e a intensidade do sinal (RSSI em dBm). Após a inserção interativa do nome da rede e da senha pelo terminal serial, o microcontrolador se conecta à internet e obtém um endereço IP.

Com a rede ativa, o sistema realiza a autenticação segura com o Google utilizando as credenciais da conta de serviço e a chave privada. O fluxo de funcionamento divide-se em duas etapas principais:

- **Leitura Inicial:** Logo após conectar, o ESP32 realiza uma leitura rápida das primeiras células da planilha para validar a comunicação.
- **Aquisição do Sensor:** Em paralelo ao envio, o ESP32 lê o sensor LDR por meio do **ADS1115**, conectado via protocolo I2C (SDA no GPIO21 e SCL no GPIO22, endereço `0x48`). O valor bruto de 16 bits obtido é mapeado para a faixa de 0° a 180° para o controle do servomotor. Tanto o ângulo do atuador quanto o valor bruto (*raw*) medido no sensor pelo ADC são enviados para a planilha. Nesse caso, o dado bruto serve como referência para o registro (*logging*), permitindo relacionar cada ângulo à leitura que o originou.
- **Envio Periódico:** Em seguida, o sistema entra em um ciclo de envio contínuo controlado por tempo (*millis()*). A cada 5 segundos, os dados são empacotados em formato JSON e enviados por meio de uma requisição HTTP para a planilha, adicionando uma nova linha de registros automaticamente.

2.3 Resultados e Discussão

Durante os testes práticos, o ESP32 listou com sucesso as redes próximas e permitiu o acesso à internet após a autenticação via terminal. A comunicação com a nuvem funcionou de maneira estável, permitindo ler e inserir os dados periodicamente na planilha do Google Drive sem perda de pacotes.

2.4 Resultados e Discussão

Foi possível verificar a transmissão desde o sensor até a chegada do dado na planilha do Google.

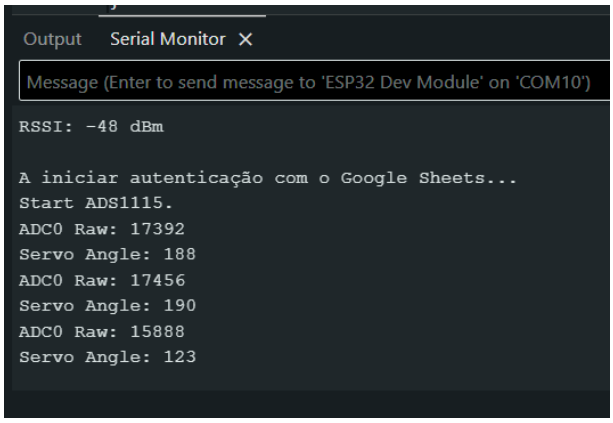


Figura 3: Monitoramento serial no ESP32

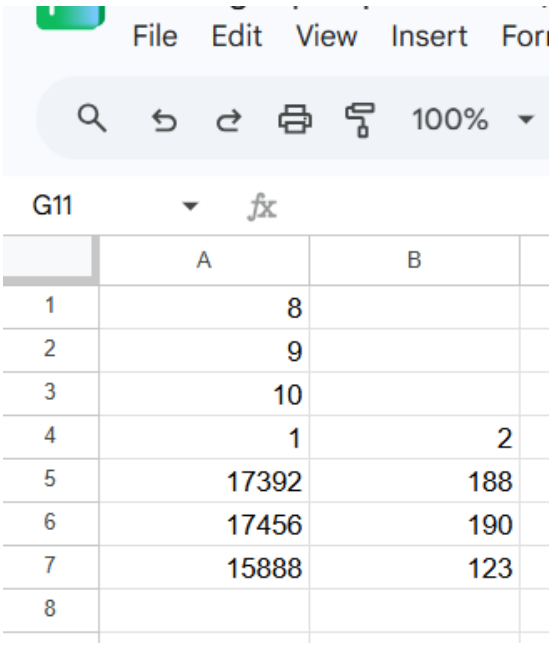


Figura 4: Dados recebidos pelo Google Sheets

3 Pós-Laboratório

3.1 Benefício de empregar um ADC externo quando o sistema contém um ESP32

Embora o ESP32 possua conversores analógico-digitais (ADCs) internos, o uso de um componente externo como o ADS1115 traz vantagens importantes:

1. **Evita conflitos com o Wi-Fi:** O segundo conversor interno do ESP32 compartilha recursos com o rádio Wi-Fi, o que pode gerar instabilidades ou bloqueios nas leituras durante o uso da rede sem fio. O ADS1115 utiliza comunicação digital (I^2C), operando de forma independente do Wi-Fi.

2. **Maior precisão e linearidade:** O conversor interno do ESP32 apresenta pequenas limitações e variações de leitura em faixas muito baixas ou muito altas de tensão. O ADS1115 oferece leituras mais estáveis, lineares e confiáveis.

3. **Maior resolução:** O ADS1115 possui resolução de 16 bits, o que permite capturar variações de sinal muito mais sutis do que os 12 bits nativos do ESP32.

3.2 Tipo de ADC empregado no ADS1115 e seu funcionamento

O ADS1115 utiliza um conversor do tipo **Delta-Sigma ($\Delta\Sigma$)**.

Ele funciona amostrando o sinal de entrada em uma frequência muito alta (processo conhecido como sobreamostragem) e convertendo-o em um fluxo rápido de bits. Em seguida, um filtro digital interno processa esse fluxo de dados, eliminando os ruídos de alta frequência e gerando uma leitura digital final de alta precisão e resolução na saída.

4 Anexos

- Arquivo `.ino` contendo o código-fonte desenvolvido na Arduino IDE.